ECON 3420A Assignment 2: Testing the CAPM, Time-Varying Beta, and Multifactor Models

Lecture Note Part 4-5 | 100 marks

LANG Ho Tin (1155209814) Jiayi Nie (1155210146) Chung Yuet Ki Toby (1155202615)

2026-11-10

(a) My predictions (for Q1–Q3)

Prediction	My answer and reason	What I later observed
Long-run beta of `GLD` vs. `AAPL` against `SPY`	`GLD` close to zero (somewhere between −0.1 and 0.2) and far below `AAPL` (about 1.1 to 1.3). Gold pays no cash flows that depend on the business cycle; its price responds to real interest rates, the dollar and safe-haven demand, so over 10 to 20 years its covariance with the market should roughly average out. Apple's earnings rise and fall with consumer spending and growth expectations, so its returns move with the market, and more than one-for-one.	As predicted. `GLD`'s beta is 0.085 with monthly returns ($t = 1.19$, not different from zero) and 0.051 with daily returns ($t = 4.01$); `AAPL`'s is 1.204 monthly and 1.079 daily, the latter just below my range. What I did not predict is that gold would still earn an alpha of 9.42% a year.
`GLD`'s beta in 2020: constant, negative, or positive?	Not constant. I expect it to turn negative in the first sell-off, when investors leave equities for safe havens, and then to turn positive later in the year, when stimulus and falling real rates lift stocks and gold together. Over a window of a few weeks, beta reflects whichever shock dominates at the time, not the long-run average.	The signs matched: -0.144 in Regime A and 0.207 in Regime C. Only the Regime C estimate is significant, though ($t = 4.153$, against -1.546 in Regime A). Beta was already negative in 2018–2019 (-0.096), so it did not "turn" negative, and I had not considered the dash-for-cash, when it was -0.008.
Does `AAPL`'s rolling beta ever turn negative?	No. It should vary (perhaps between 0.7 and 1.7) but stay positive throughout: Apple's profits depend on the same macroeconomic conditions as the rest of the market, and Apple is itself one of the largest stocks in the index.	Correct. The 60-day beta was positive on 100.0% of trading days from 2016 to 2025. Its range, from 0.506 (30 January 2025) to 2.024 (13 October 2020), is wider than the 0.7 to 1.7 I expected.

Prediction recorded on: 10 November 2026, 23:04 HKT (before any regression was run)

(b) My predictions (for Q4–Q5)

ETF	Predicted signs of β_{SMB} and β_{HML}, and reason	What I later observed
`IWD` (large value)	$\beta_{\text{SMB}} < 0$ (small in size), $\beta_{\text{HML}} > 0$. It holds the large-cap stocks that form the short leg of SMB, and high book-to-market stocks that form the long leg of HML. Because the Russell 1000 is close to the market itself, the SMB loading should be only slightly negative.	$\hat{\beta}_{\text{SMB}} = -0.02$ $(t = -0.49)$, $\hat{\beta}_{\text{HML}} = 0.37$ $(t = 15.81)$. Signs as predicted; the SMB loading is statistically zero.
`IWF` (large growth)	$\beta_{\text{SMB}} < 0, \beta_{\text{HML}} < 0$. Large caps again, but now the low book-to-market stocks that form the short leg of HML.	$\hat{\beta}_{\text{SMB}} = -0.13$ $(t = -4.50)$, $\hat{\beta}_{\text{HML}} = -0.32$ $(t = -17.13)$. Both as predicted and significant.
`IWN` (small value)	$\beta_{\text{SMB}} > 0$ (large, near 0.8 to 1), $\beta_{\text{HML}} > 0$. Russell 2000 stocks sit in the long leg of SMB, and the value half sits in the long leg of HML.	$\hat{\beta}_{\text{SMB}} = 0.82$ $(t = 30.77)$, $\hat{\beta}_{\text{HML}} = 0.55$ $(t = 30.08)$. Both as predicted, with the SMB loading inside my range.
`IWO` (small growth)	$\beta_{\text{SMB}} > 0$ (large), $\beta_{\text{HML}} < 0$. Small caps load on SMB; as growth stocks with low book-to-market they should co-move with the short leg of HML.	$\hat{\beta}_{\text{SMB}} = 0.89$ $(t = 22.45)$ as predicted. $\hat{\beta}_{\text{HML}} = -0.05$ $(t = -1.74)$ has the predicted sign but is not different from zero: the clearly negative value loading I expected is missing (Q5 returns to this).

General reasoning: a fund's factor loading reflects where its holdings sit relative to the factor's long and short legs, so the ETF labels (size and style) should map directly into the signs.

Prediction recorded on: 10 November 2026, 23:04 HKT (before any regression was run)

Environment Setup Guide Codes

Configuration

Tickers and sample windows from the assignment. Change a window only if you explain why in (c).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

config = {
    "market": "SPY",
    "q1_assets": ["GLD", "GC=F", "GDX", "AAPL"],
```

```
    "q1_start": "2005-01-01",  # or "2016-01-01"
    "q1_end": "2025-12-31",
    "q2_assets": ["GLD", "GDX", "AAPL"],
    "q3_assets": ["GLD", "AAPL"],
    "q3_start": "2016-01-01",
    "q3_end": "2025-12-31",
    "q4_assets": ["IWD", "IWF", "IWN", "IWO", "AAPL", "GLD"],
    "q5_assets": ["IWM", "IJR", "ARKK", "QUAL", "IWO"],
    "ff_start": "2016-01",
    "ff_end": "2025-12",
}

regimes = {  # Q2 windows (inclusive)
    "Pre-COVID 2018-2019": ("2018-01-01", "2019-12-31"),
    "Regime A": ("2020-01-02", "2020-03-06"),
    "Regime B": ("2020-03-09", "2020-03-23"),
    "Regime C": ("2020-03-24", "2020-12-31"),
    "Full-year 2020": ("2020-01-01", "2020-12-31"),
}
```

Table helper

`show_table` prints a table in both HTML and PDF.

1. Long-run beta of gold vs. an operating stock (20 marks)

$$X_{i,t} = \alpha_i + \beta_i X_{M,t} + e_{i,t}, \qquad X_{i,t} = R_{i,t} - R_{f,t}, \quad X_{M,t} = R_{M,t} - R_{f,t}.$$

Task to AI:

```
import yfinance as yf
from pathlib import Path

DATA_DIR = Path("data")  # downloads are cached here so every render uses the same data
DATA_DIR.mkdir(exist_ok=True)


def get_prices(tickers, start, end):
    """Daily adjusted closing prices from Yahoo Finance for start..end (inclusive)."""
    key = "_".join(tickers).replace("=", "").replace("^", "")
    path = DATA_DIR / f"yahoo_{key}_{start}_{end}.csv"
    if path.exists():
        return pd.read_csv(path, index_col=0, parse_dates=True)
    day_after = pd.Timestamp(end) + pd.Timedelta(days=1)  # yfinance excludes `end`
    px = yf.download(tickers, start=start, end=day_after, auto_adjust=True,
                     progress=False)["Close"]
    px = px[tickers].dropna(how="all")
    if px.empty or px.isna().all().any():
        raise RuntimeError(f"Yahoo Finance returned no data for some of {tickers}")
    px.index = pd.DatetimeIndex(px.index).tz_localize(None)
    px.index.name = "Date"
    px.to_csv(path)
    return px
```

```python
def excess_returns(prices, rf_annual, periods_per_year):
    """Simple returns less R_f/periods_per_year (yield quoted at the period's start)."""
    rf = rf_annual.shift(1) / periods_per_year
    return prices.pct_change(fill_method=None).sub(rf, axis=0)


def ols(y, X):
    """OLS of y on a constant and X, using the rows where every variable is observed."""
    data = pd.concat([y, X], axis=1).dropna()
    return sm.OLS(data.iloc[:, 0], sm.add_constant(data.iloc[:, 1:])).fit()


market = config["market"]
q1_tickers = [market] + config["q1_assets"]
q1_start, q1_end = config["q1_start"], config["q1_end"]

# One extra month before the sample so that the first daily and monthly returns exist
download_start = (pd.Timestamp(q1_start) - pd.DateOffset(months=1)).strftime("%Y-%m-%d")
raw = get_prices(q1_tickers + ["^IRX"], download_start, q1_end)
raw = raw.loc[raw[market].notna()]  # keep SPY's (NYSE) trading days

prices = raw[q1_tickers]
rf_annual = raw["^IRX"].ffill() / 100  # 3-month T-bill yield: annualized, in decimals

daily_returns = prices.pct_change(fill_method=None).loc[q1_start:q1_end]
daily_excess = excess_returns(prices, rf_annual, 252).loc[q1_start:q1_end]

month_end_prices = prices.resample("ME").last()
month_end_rf = rf_annual.resample("ME").last()
monthly_excess = excess_returns(month_end_prices, month_end_rf, 12).loc[q1_start:q1_end]

coverage = pd.DataFrame({
    "First daily return": daily_excess.apply(lambda s: s.first_valid_index().date()),
    "Last daily return": daily_excess.apply(lambda s: s.last_valid_index().date()),
    "Daily obs.": daily_excess.count(),
    "Monthly obs.": monthly_excess.count(),
})
coverage.index.name = "Ticker"
show_table(coverage)
```

Ticker	First daily return	Last daily return	Daily obs.	Monthly obs.
SPY	2005-01-03	2025-12-31	5283	252
GLD	2005-01-03	2025-12-31	5283	252
GC=F	2005-01-04	2025-12-31	5264	252
GDX	2006-05-23	2025-12-31	4934	235
AAPL	2005-01-03	2025-12-31	5283	252

```
frequencies = {"Monthly": (monthly_excess, 12, "%Y-%m"),
               "Daily": (daily_excess, 252, "%Y-%m-%d")}


def single_index(asset, excess):
    """OLS of an asset on SPY over the asset's own sample (pairwise with SPY)."""
    sample = excess[[asset, market]].dropna()
    return ols(sample[asset], sample[market]), sample


# Summary tables: means, volatilities and alphas are annualized, in percent per year
for frequency, (excess, periods, date_format) in frequencies.items():
    columns = {}
    for asset in config["q1_assets"]:
        fit, sample = single_index(asset, excess)
        first, last = sample.index[[0, -1]].strftime(date_format)
        x = sample[asset]
        columns[asset] = {
            "Sample start": first,
            "Sample end": last,
            "$n$": int(fit.nobs),
            "$E(X_i)$, % p.a.": f"{100 * periods * x.mean():.2f}",
            r"$\sigma_i$, % p.a.": f"{100 * np.sqrt(periods) * x.std():.2f}",
            r"$\hat\beta_i$": fit.params[market],
            r"SE($\hat\beta_i$)": fit.bse[market],
            r"$t$ for $H_0\colon\beta_i=0$": f"{fit.tvalues[market]:.2f}",
            "$R^2$": fit.rsquared,
            r"$\hat\alpha_i$, % p.a.": f"{100 * periods * fit.params['const']:.2f}",
            r"$t$ for $H_0\colon\alpha_i=0$": f"{fit.tvalues['const']:.2f}",
        }
    show_table(pd.DataFrame(columns).rename_axis(frequency))

# Var(X_i) = beta_i^2 Var(X_M) + Var(e_i), annualized, in %² (square root = sigma in %)
columns = {}
for asset in ["GLD", "AAPL"]:
    for frequency, (excess, periods, _) in frequencies.items():
        fit, sample = single_index(asset, excess)
        beta, scale = fit.params[market], 1e4 * periods
        total = scale * sample[asset].var()
        market_variance = scale * sample[market].var()
        specific = scale * fit.resid.var()
        market_premium = 100 * periods * sample[market].mean()
        columns[f"{asset} {frequency.lower()}"] = {
            "Var($X_M$), %²": market_variance,
            "Var($X_i$), %²": total,
            r"$\hat\beta_i^2$Var($X_M$), %²": beta**2 * market_variance,
            "Var($e_i$), %²": specific,
            "Market share, %": 100 * beta**2 * market_variance / total,
            "Specific share, %": 100 * specific / total,
            "$E(X_M)$, % p.a.": market_premium,
            r"$\hat\beta_i E(X_M)$, % p.a.": beta * market_premium,
```

```
        }
show_table(pd.DataFrame(columns).rename_axis("Decomposition"), 1)
```

Monthly	GLD	GC=F	GDX	AAPL
Sample start	2005-01	2005-01	2006-06	2005-01
Sample end	2025-12	2025-12	2025-12	2025-12
n	252	252	235	252
$E(X_i)$, % p.a.	10.23	10.67	10.02	30.26
σ_i, % p.a.	16.65	16.60	37.02	30.83
$\hat{\beta}_i$	0.085	0.083	0.563	1.204
SE($\hat{\beta}_i$)	0.071	0.071	0.155	0.107
t for $H_0 : \beta_i = 0$	1.19	1.18	3.63	11.21
R^2	0.006	0.006	0.053	0.335
$\hat{\alpha}_i$, % p.a.	9.42	9.87	4.35	18.77
t for $H_0 : \alpha_i = 0$	2.55	2.68	0.52	3.36

Daily	GLD	GC=F	GDX	AAPL
Sample start	2005-01-03	2005-01-04	2006-05-23	2005-01-03
Sample end	2025-12-31	2025-12-31	2025-12-31	2025-12-31
n	5283	5264	4934	5283
$E(X_i)$, % p.a.	10.38	10.93	11.88	30.39
σ_i, % p.a.	17.67	17.88	40.88	32.16
$\hat{\beta}_i$	0.051	0.023	0.570	1.079
SE($\hat{\beta}_i$)	0.013	0.013	0.029	0.018
t for $H_0 : \beta_i = 0$	4.01	1.81	19.87	60.41
R^2	0.003	0.001	0.074	0.409
$\hat{\alpha}_i$, % p.a.	9.86	10.69	5.71	19.36
t for $H_0 : \alpha_i = 0$	2.56	2.73	0.64	3.58

Decomposition	GLD monthly	GLD daily	AAPL monthly	AAPL daily
Var(X_M), %²	219.6	363.1	219.6	363.1
Var(X_i), %²	277.4	312.1	950.6	1034.0
$\hat{\beta}_i^2$Var(X_M), %²	1.6	0.9	318.2	422.5
Var(e_i), %²	275.8	311.2	632.4	611.4
Market share, %	0.6	0.3	33.5	40.9
Specific share, %	99.4	99.7	66.5	59.1
$E(X_M)$, % p.a.	9.5	10.2	9.5	10.2
$\hat{\beta}_i E(X_M)$, % p.a.	0.8	0.5	11.5	11.0

The first two tables report the single-index regression for each asset on monthly and on daily excess returns, each over the asset's own sample. Means, volatilities and alphas are annualized ($\times$ 12 or $\times$ 252; volatility $\times$ $\sqrt{12}$ or $\times$ $\sqrt{252}$) and standard errors are classical OLS. The third table splits the annualized variance of GLD and AAPL, in %² so that its square root is the volatility in percent, into a market part and an asset-specific part; its last row is the premium the CAPM implies, $\hat{\beta}_i E(X_M)$.

Task to Student:

- **Is gold's beta different from zero?** I reject $H_0 : \beta_i = 0$ at the 5% level when $|t| > 1.96$. With monthly returns I cannot reject it for either gold series: GLD has $\hat{\beta} = 0.085$ ($t = 1.19$) and GC=F has 0.083 ($t = 1.18$). With daily returns I reject it for GLD ($\hat{\beta} = 0.051$, $t = 4.01$) but not for GC=F (0.023, $t = 1.81$, significant only at the 10% level). The two frequencies therefore disagree for GLD, even though the daily estimate is the smaller one. What changes is precision. With 5,283 observations instead of 252, the standard error falls from 0.071 to 0.013, so a beta that is economically negligible ($R^2 = 0.003$) becomes statistically detectable; the monthly 95% interval (about -0.05 to 0.22) contains the daily estimate. Daily data also add timing effects: gold futures settle before the NYSE close, which may explain why the daily beta of GC=F is less than half of GLD's while their monthly betas are almost equal. AAPL is the opposite case: $\hat{\beta} = 1.204$ ($t = 11.21$) monthly and 1.079 ($t = 60.41$) daily, with R^2 of 0.335 and 0.409.
- **Does $\beta \approx 0$ mean no risk?** No. In monthly data GLD's volatility is 16.65% a year, and its variance of 277.4 %² exceeds the market's 219.6, yet its R^2 is only 0.006. That variance splits into a market part $\hat{\beta}_i^2 \mathrm{Var}(X_M) = 1.6$ and an asset-specific part $\mathrm{Var}(e_i) = 275.8$: 0.6% systematic and 99.4% idiosyncratic. For AAPL, 950.6 splits into 318.2 and 632.4, so 33.5% is systematic. Daily data give the same picture (0.3% against 40.9%). $\mathrm{Var}(X_i) = 0$ would need both parts to be zero; a zero beta removes only the first. The CAPM prices only the first: an investor who holds the market portfolio diversifies asset-specific risk away, so only covariance with the market adds to portfolio risk and earns a premium. With $\hat{\beta} = 0.085$ and a market premium of 9.5%, gold's required premium is $\hat{\beta}_i E(X_M) = 0.8\%$ a year, hence $E(R_{\mathrm{GLD}}) \approx R_f$. Realized returns did not match this: GLD earned 10.23% a year above bills, with $\hat{\alpha} = 9.42\%$ ($t = 2.55$, so $\alpha = 0$ is rejected at the 5% level). Either 2005 to 2025 was an unexpectedly good period for gold, or the single-factor CAPM misses what drives its return.
- **Gold, gold miners and Apple.** The betas rank GLD < GDX < AAPL at both frequencies: 0.085, 0.563 and 1.204 with monthly returns, and 0.051, 0.570 and 1.079 with daily returns (GDX over its shorter sample, from mid-2006). GDX's beta is clearly different from zero ($t = 3.63$ monthly, 19.87 daily) but only about half of Apple's, and its monthly R^2 is just 0.053, so most of its 37.02% volatility is still not market risk. Miners have a higher beta than the metal because GDX holds shares in operating companies, not gold. A miner's profit is the gold price minus extraction costs, and largely fixed costs (operating leverage) and debt (financial leverage) magnify every shock to the value of its assets, which is why GDX is more than twice as volatile as GLD. Leverage alone would only scale up gold's small beta. The rest comes from the business: energy, labour and financing costs move with the economy, and miners' shares fall with other equities in market-wide sell-offs. Apple's beta is higher still because its cash flows depend directly on consumer spending.

2. Subperiod analysis around the 2020 COVID-19 pandemic (20 marks)

Task to AI:

Beta is a regression slope, so it needs several return observations inside the window. Regime A covers 45 trading days but only 2 month-ends, and Regime B covers 11 trading days and no month-end. Two monthly returns fit a line exactly, leaving no degrees of freedom for $SE(\hat{\beta})$, a t-test or a meaningful R^2, and Regime B contains no monthly return at all. March 2020 also spans all three regimes. Daily returns give 45 and 11 observations.

In the first table, t tests $H_0 : \beta_i = 0$ and "t crit." is the two-sided 5% Student-t critical value with $n - 2$ degrees of freedom. In the second, cumulative returns compound the raw daily returns over each window's trading days ("Days").

```python
from scipy import stats


def scl_stats(excess, asset):
    """SCL regression of an asset on the market, with the t-test of beta = 0."""
    fit = ols(excess[asset], excess[market])
    n = int(fit.nobs)
    return {
        "Asset": asset,
        "n": n,
        r"$\hat\beta$": fit.params[market],
        "SE": fit.bse[market],
        "t": fit.tvalues[market],
        "t crit.": stats.t.ppf(0.975, n - 2),  # two-sided 5% test, n - 2 d.f.
        r"$\hat\rho$": excess[asset].corr(excess[market]),
        "$R^2$": fit.rsquared,
    }


weekly_prices = prices.resample("W-FRI").last()  # Friday-ending weeks
weekly_excess = excess_returns(weekly_prices, rf_annual.resample("W-FRI").last(), 52)

samples = {(name, "daily"): daily_excess.loc[a:b] for name, (a, b) in regimes.items()}
year_start, year_end = regimes["Full-year 2020"]
samples["Full-year 2020", "weekly"] = weekly_excess.loc[year_start:year_end]

rows = []
for (name, frequency), sample in samples.items():
    rows.append({"Asset": f"**{name}, {frequency}**"})  # heading row for the window
    rows += [scl_stats(sample, asset) for asset in config["q2_assets"]]
regime_table = pd.DataFrame(rows, dtype=object).fillna("").set_index("Asset")
show_table(regime_table.rename_axis(None))

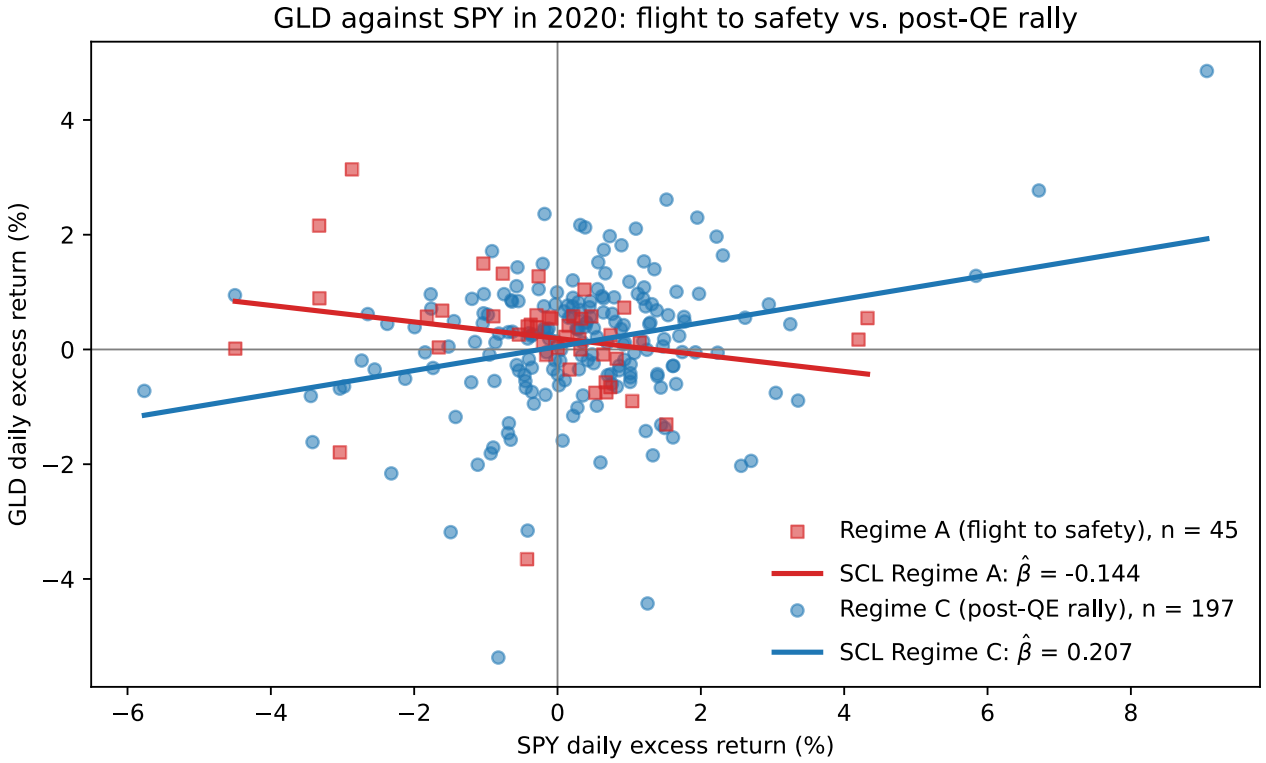
window_rows = {}
for name, (a, b) in regimes.items():
    window = daily_returns.loc[a:b, [market] + config["q2_assets"]]
    window_rows[name] = {
        "Days": len(window),
        "Month-ends": len(pd.date_range(a, b, freq="ME")),
        **(100 * ((1 + window).prod() - 1)),
    }
cumulative = pd.DataFrame.from_dict(window_rows, orient="index")
show_table(cumulative.rename_axis("Cumulative return (%)"), decimals=1)
```

	n	$\hat{\beta}$	SE	t	t crit.	$\hat{\rho}$	R^2
Pre-COVID 2018-2019, daily							
GLD	503	−0.096	0.032	−3.004	1.965	−0.133	0.018
GDX	503	−0.125	0.078	−1.593	1.965	−0.071	0.005
AAPL	503	1.366	0.055	24.780	1.965	0.742	0.551
Regime A, daily							
GLD	45	−0.144	0.093	−1.546	2.017	−0.230	0.053
GDX	45	0.199	0.215	0.927	2.017	0.140	0.020
AAPL	45	1.488	0.099	15.004	2.017	0.916	0.840
Regime B, daily							
GLD	11	−0.008	0.124	−0.063	2.262	−0.021	0.000
GDX	11	0.047	0.652	0.072	2.262	0.024	0.001
AAPL	11	1.132	0.084	13.476	2.262	0.976	0.953
Regime C, daily							
GLD	197	0.207	0.050	4.153	1.972	0.285	0.081
GDX	197	0.654	0.121	5.387	1.972	0.360	0.130
AAPL	197	1.133	0.077	14.695	1.972	0.725	0.525
Full-year 2020, daily							
GLD	253	0.093	0.036	2.571	1.969	0.160	0.026
GDX	253	0.391	0.110	3.558	1.969	0.219	0.048
AAPL	253	1.158	0.049	23.481	1.969	0.829	0.687
Full-year 2020, weekly							
GLD	52	0.309	0.084	3.662	2.009	0.460	0.212
GDX	52	0.829	0.225	3.681	2.009	0.462	0.213
AAPL	52	0.972	0.108	8.974	2.009	0.785	0.617

Cumulative return (%)	Days	Month-ends	SPY	GLD	GDX	AAPL
Pre-COVID 2018-2019	503	24	25.2	15.6	27.6	78.8
Regime A	45	2	−7.6	10.3	0.4	−1.3
Regime B	11	0	−24.6	−7.1	−25.4	−22.4
Regime C	197	10	69.8	21.9	65.1	138.0
Full-year 2020	253	12	18.3	24.8	23.7	82.3

```
styles = {  # regime: (description, colour, marker, drawing order)
    "Regime A": ("flight to safety", "tab:red", "s", 3),
    "Regime C": ("post-QE rally", "tab:blue", "o", 2),
}
fig, ax = plt.subplots(figsize=(9, 5))
for name, (description, color, marker, zorder) in styles.items():
    sample = 100 * samples[name, "daily"]  # in percent
```

```
    x, y = sample[market], sample["GLD"]
    fit = ols(y, x)
    beta = fit.params[market]
    ends = np.array([x.min(), x.max()])
    ax.scatter(x, y, s=28, alpha=0.55, color=color, marker=marker, zorder=zorder,
               label=f"{name} ({description}), n = {len(x)}")
    ax.plot(ends, fit.params["const"] + beta * ends, color=color, lw=2.2, zorder=4,
            label=rf"SCL {name}: $\hat\beta$ = {beta:.3f}")
ax.axhline(0, color="grey", lw=0.8)
ax.axvline(0, color="grey", lw=0.8)
ax.set_xlabel("SPY daily excess return (%)")
ax.set_ylabel("GLD daily excess return (%)")
ax.set_title("GLD against SPY in 2020: flight to safety vs. post-QE rally")
ax.legend(frameon=False)
plt.show()
```



Task to Student:

- **Prediction against the estimates.** I predicted that GLD's beta in 2020 would not be constant: negative in the first sell-off, then positive later in the year. The signs match: $\hat{\beta}_{\text{GLD}}$ is -0.144 in Regime A and 0.207 in Regime C, and gold gained 10.3% in Regime A while SPY lost 7.6%. The prediction missed in three ways. First, the Regime A estimate is not statistically different from zero ($t = -1.546$), so the data support the negative sign only weakly. Second, beta did not "turn" negative: it was already -0.096 in 2018-2019, so Regime A continued the baseline. Third, I said nothing about the dash-for-cash episode, in which gold's beta was -0.008 and gold fell 7.1% while SPY fell 24.6%: it lost less than equities but paid nothing as a safe haven. My remark that a short-window beta reflects the dominant shock holds up, because the full-year daily beta of 0.093 averages over three different regimes. AAPL's beta stayed positive as predicted (1.488, 1.132, 1.133), although Regime A lies above my 1.1 to 1.3 range.
- **Significance and the change of sign.** I reject $H_0 : \beta_i = 0$ at the 5% level when $|t|$ exceeds the Student-t critical value with $n - 2$ degrees of freedom. Only Regime C passes ($t = 4.153$ against 1.972, $n = 197$);

Regime A ($t = -1.546$ against 2.017, $n = 45$) and Regime B ($t = -0.063$ against 2.262, $n = 11$) do not. Fewer days raise both $SE(\hat{\beta})$ (0.093 and 0.124 against 0.050 in Regime C) and the critical value, so with 11 days "near zero" means that I cannot tell, not that beta is zero. The sign follows the type of shock. A growth/fear shock (Regime A) cuts expected cash flows and raises safe-haven demand, so gold rises when stocks fall: negative covariance with the market. A margin spiral (Regime B) forces leveraged investors to sell every liquid asset, gold included, which cancels the hedge. A monetary shock (Regime C) lowers real interest rates, which lifts both equity valuations and gold, which pays no yield: positive covariance. `AAPL`'s cash flows and discount rate respond to all three shocks as the market's do, so its beta never changes sign. It exceeds 1 in every regime (1.488, 1.132, 1.133), though only Regime A's estimate is more than two standard errors above 1.

- **Negative beta and the required return.** With a positive market risk premium, $\beta_i < 0$ makes $\beta_i(\mathbb{E}(R_M) - R_f)$ negative, so the CAPM requires $\mathbb{E}(R_i) < R_f$. Gold's baseline beta of -0.096 (significant, $t = -3.004$) puts its required return below R_f by 9.6% of the market risk premium. Mean-variance investors accept this because they judge an asset by what it adds to the variance of the market portfolio, which is its covariance with that portfolio, not by its own volatility. An asset with negative covariance tends to pay off when the rest of the portfolio loses, as `GLD` did in Regime A (+10.3% against −7.6% for `SPY`), so holding it lowers portfolio variance beyond ordinary diversification. It works like insurance: investors bid up its price until its expected return is below R_f, and the shortfall is the insurance premium. My table adds two caveats. The hedge is small (R^2 of 0.018 in 2018-2019 and 0.053 in Regime A, so almost all of gold's variance is unsystematic), and it is unreliable: in Regime B gold fell 7.1% with a beta of -0.008.

3. Visualizing time-varying beta with rolling regressions (20 marks)

$$\hat{\beta}_{i,t}^{(L)} = \frac{\widehat{\mathbf{Cov}}_{t-L+1:t}(X_i, X_M)}{\widehat{\mathbf{Var}}_{t-L+1:t}(X_M)}.$$

Task to AI:

```
q3_assets = config["q3_assets"]
q3_excess = daily_excess.loc[config["q3_start"]:config["q3_end"]]
x_mkt, x_assets = q3_excess[market], q3_excess[q3_assets]


def rolling_ols(window):
    """Rolling beta = Cov(X_i, X_M) / Var(X_M), and its OLS standard error."""
    var_mkt = x_mkt.rolling(window).var()
    beta = x_assets.rolling(window).cov(x_mkt).div(var_mkt, axis=0)
    # SE^2 = residual variance / sum of squares of X_M
    #      = (Var(X_i) / Var(X_M) - beta^2) / (L - 2)
    var_ratio = x_assets.rolling(window).var().div(var_mkt, axis=0)
    se = np.sqrt((var_ratio - beta**2) / (window - 2))
    return beta.dropna(), se.dropna()


def full_sample(asset):
    """Constant beta: one regression on the whole 2016-2025 daily sample."""
    fit = ols(x_assets[asset], x_mkt)
    return {"Full-sample beta": fit.params[market], "SE": fit.bse[market]}


q3_beta, q3_se = {}, {}
for window in (60, 252):
```

```
    q3_beta[window], q3_se[window] = rolling_ols(window)
q3_full = pd.DataFrame({asset: full_sample(asset) for asset in q3_assets}).T

q3_windows = pd.DataFrame(
    [{"Window": f"{window}-day", "First beta": beta.index[0].date(),
      "Last beta": beta.index[-1].date(), "Betas per asset": len(beta)}
     for window, beta in q3_beta.items()])
show_table(q3_windows.set_index("Window"))
```

Window	First beta	Last beta	Betas per asset
60-day	2016-03-30	2025-12-31	2455
252-day	2016-12-30	2025-12-31	2263

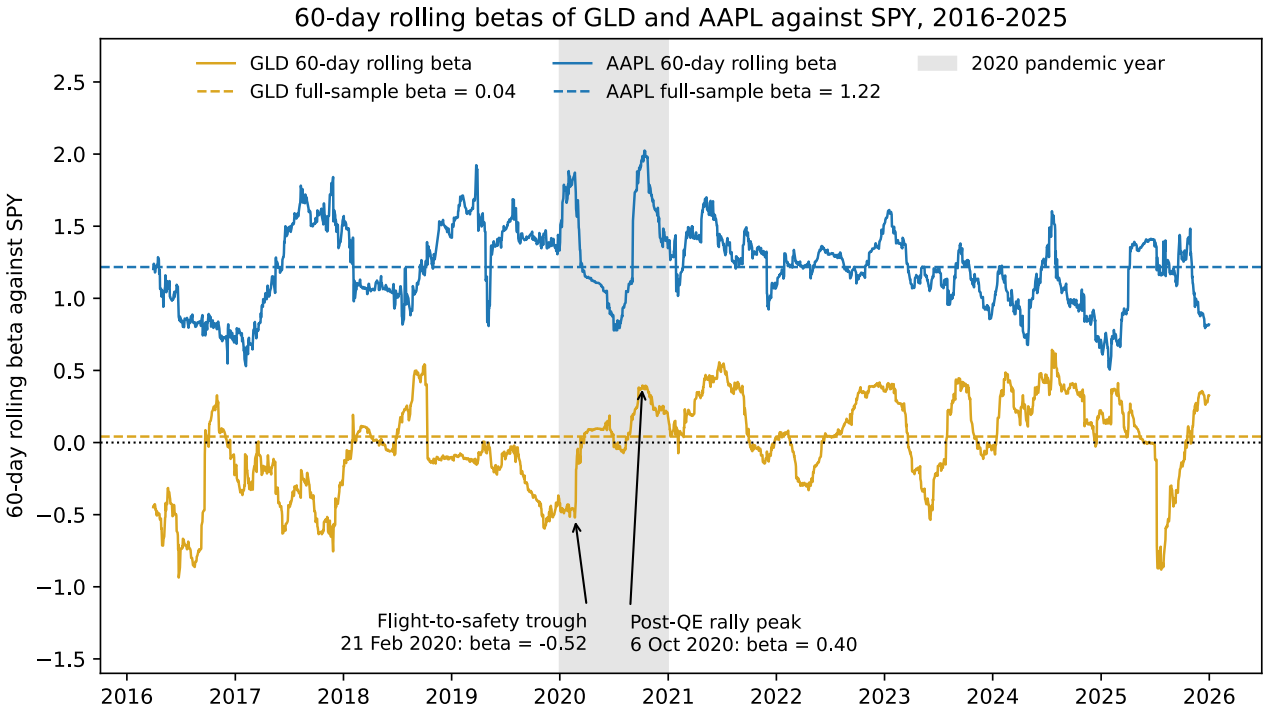
```
colors = {"GLD": "goldenrod", "AAPL": "tab:blue"}
beta_60, full_beta = q3_beta[60], q3_full["Full-sample beta"]

# Gold's 2020 extremes: the lowest 60-day beta of the year and the highest after it
gld_2020 = beta_60.loc["2020", "GLD"]
trough = gld_2020.idxmin()
peak = gld_2020.loc[trough:].idxmax()

fig, ax = plt.subplots(figsize=(10, 5.5))
for asset in q3_assets:
    ax.plot(beta_60[asset], color=colors[asset], lw=1.2,
            label=f"{asset} 60-day rolling beta")
    ax.axhline(full_beta[asset], color=colors[asset], ls="--", lw=1.2,
               label=f"{asset} full-sample beta = {full_beta[asset]:.2f}")
ax.axhline(0, color="black", ls=":", lw=1)
ax.axvspan(pd.Timestamp("2020-01-01"), pd.Timestamp("2020-12-31"), color="grey",
           alpha=0.2, label="2020 pandemic year")

# Labels go below the gold line, to the left and right of the points they mark
marks = [("Flight-to-safety trough", trough, "right", (6, -45)),
         ("Post-QE rally peak", peak, "left", (-6, -109))]
for name, date, side, offset in marks:
    ax.annotate(f"{name}\n{date.day} {date:%b %Y}: beta = {gld_2020[date]:.2f}",
                xy=(date, gld_2020[date]), xytext=offset,
                textcoords="offset points", ha=side, va="top", fontsize=9,
                arrowprops={"arrowstyle": "->", "relpos": (side == "right", 1)})

ax.set_ylim(-1.6, 2.8)
ax.set_ylabel("60-day rolling beta against SPY")
ax.set_title("60-day rolling betas of GLD and AAPL against SPY, 2016-2025")
ax.legend(ncol=3, loc="upper center", fontsize=9, frameon=False)
plt.show()
```




```python
def describe(asset, window):
    beta = q3_beta[window][asset]
    return {
        "Asset": asset,
        "Window": f"{window}-day",
        "Min": beta.min(),
        "Date of min": beta.idxmin().date(),
        "Max": beta.max(),
        "Date of max": beta.idxmax().date(),
        "% days > 0": f"{100 * (beta > 0).mean():.1f}",
    }


rows = [describe(asset, window) for asset in q3_assets for window in q3_beta]
show_table(pd.DataFrame(rows).set_index("Asset"))

# Precision of the three estimators: rolling SEs are averaged over the sample
precision = q3_full.assign(**{f"Mean SE, {window}-day": q3_se[window].mean()
                              for window in (252, 60)})
show_table(precision.rename_axis("Asset"))
```

Asset	Window	Min	Date of min	Max	Date of max	% days > 0
GLD	60-day	−0.936	2016-06-24	0.643	2024-07-19	51.6
GLD	252-day	−0.479	2017-02-02	0.349	2024-11-04	59.7
AAPL	60-day	0.506	2025-01-30	2.024	2020-10-13	100.0
AAPL	252-day	0.809	2017-04-17	1.585	2021-07-19	100.0

Asset	Full-sample beta	SE	Mean SE, 252-day	Mean SE, 60-day
GLD	0.042	0.016	0.056	0.132
AAPL	1.217	0.021	0.078	0.182

Task to Student:

- **AAPL versus GLD.** AAPL's 60-day beta varies widely, from 0.506 (30 January 2025) to 2.024 (13 October 2020) around its full-sample value of 1.217, but it is positive on 100.0% of trading days, and its line stays above GLD's throughout. GLD's 60-day beta ranges from -0.936 (24 June 2016, the day the Brexit result was announced) to 0.643 (19 July 2024) and is positive on only 51.6% of days: its sign changes, not only its size. Apple's exposure keeps its sign because of where its value comes from. Its cash flows depend on household and business spending, which rises and falls with the economy, and its shares are priced with the same discount rates and risk appetite as other stocks; it is also one of the largest holdings in SPY, so its own return is part of the market's. What varies over time is the degree of sensitivity (product cycles, valuation), so beta moves but stays positive. Gold pays no cash flows; its price responds to real interest rates, the dollar and safe-haven demand, which can move it with or against stocks.
- **Window length.** The shorter the window, the wider the range of estimates. GLD's 60-day beta runs from -0.936 to 0.643, its 252-day beta from -0.479 to 0.349, and the full sample gives a single 0.042; for AAPL the ranges are 0.506 to 2.024 and 0.809 to 1.585, around 1.217. The choice is a variance-bias trade-off. The standard error of an estimated beta shrinks roughly with $1/\sqrt{L}$: on average it is 0.132 for GLD (0.182 for AAPL) at $L = 60$, 0.056 (0.078) at $L = 252$, and 0.016 (0.021) in the full sample. A 60-day beta is therefore noisy, and its small moves are estimation error, but it responds quickly when the true beta changes; GLD's extremes are about five to seven average standard errors from zero, so its sign changes are real. A long window is precise, but when beta is time-varying it averages over different regimes and gives a biased, lagging estimate of the current beta: GLD's 252-day minimum is about half the size of the 60-day one and is dated more than seven months later (2 February 2017), and the full-sample 0.042 hides every swing.
- **Assuming a constant beta for gold.** Portfolio beta is the weighted average of the asset betas, so a wrong beta for gold misstates systematic risk. On average $\beta = 0$ looks right (the full-sample beta is 0.042), but my plot shows long spells far from zero. Gold's 60-day beta was -0.52 on 21 February 2020, which summarises the 60 trading days before the crash. Once the March liquidity squeeze entered the window, with investors selling gold too to raise cash, it jumped to about zero within three weeks: the hedge was gone while stocks were still falling. Under rate cuts and QE, stocks and gold rose together, and the beta reached 0.40 on 6 October 2020; its sample maximum, 0.643 on 19 July 2024, came as markets priced in Fed rate cuts. With 80% in SPY and 20% in GLD, portfolio beta is 0.80 if gold's beta is 0 and 0.70 if it is -0.52, but 0.88 at 0.40 and 0.93 at 0.643. Assuming $\beta = 0$ understates market risk in an easing cycle; assuming $\beta < 0$ overstates the protection in a liquidity crisis. Such time variation in β_i is what makes the CAPM hard to test.

4. From one factor to three: the Fama-French three-factor model (20 marks)

$$X_{i,t} = \alpha_i + \beta_{i,M} X_{M,t} + \beta_{i,\mathrm{SMB}} \mathrm{SMB}_t + \beta_{i,\mathrm{HML}} \mathrm{HML}_t + e_{i,t}.$$

Task to AI:

```
import io
import zipfile
import requests

FF_URL = "https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/"
ff_start, ff_end = config["ff_start"], config["ff_end"]
```

```
def get_ff_factors(name):
    """Monthly factors from a Ken French Data Library file, in decimals."""
    path = DATA_DIR / f"{name}.csv"
    if not path.exists():
        response = requests.get(f"{FF_URL}{name}_CSV.zip", timeout=60,
                                headers={"User-Agent": "Mozilla/5.0"})
        response.raise_for_status()
        with zipfile.ZipFile(io.BytesIO(response.content)) as archive:
            path.write_bytes(archive.read(archive.namelist()[0]))
    lines = path.read_text(encoding="latin-1").splitlines()
    header = next(i for i, line in enumerate(lines) if line.startswith(",Mkt-RF"))
    n_months = 0  # the monthly block ends at the first row whose key is not YYYYMM
    for line in lines[header + 1:]:
        key = line.split(",")[0].strip()
        if not (key.isdigit() and len(key) == 6):
            break
        n_months += 1
    factors = pd.read_csv(path, skiprows=header, nrows=n_months, index_col=0,
                          encoding="latin-1")
    months = pd.to_datetime(factors.index.astype(str), format="%Y%m")
    factors.index = months.to_period("M").rename("Month")
    factors.columns = factors.columns.str.strip()
    return factors / 100  # the files are in percent


def monthly_excess_ff(tickers, rf):
    """Month-end simple returns minus the Ken French RF, from ff_start to ff_end."""
    # start one month early so that the first month's return exists
    first_day = (pd.Period(ff_start, "M") - 1).start_time.strftime("%Y-%m-%d")
    last_day = pd.Period(ff_end, "M").end_time.strftime("%Y-%m-%d")
    month_end = get_prices(tickers, first_day, last_day).resample("ME").last()
    returns = month_end.pct_change(fill_method=None).to_period("M")
    return returns.sub(rf, axis=0).loc[ff_start:ff_end, tickers]


ff3_all = get_ff_factors("F-F_Research_Data_Factors")
ff3 = ff3_all.loc[ff_start:ff_end]
q4_excess = monthly_excess_ff(config["q4_assets"], ff3_all["RF"])

print(f"Ken French 3-factor file: {ff3_all.index[0]} to {ff3_all.index[-1]}")
print(f"Sample used: {ff3.index[0]} to {ff3.index[-1]} ({len(ff3)} months), "
      f"months with a missing ETF return: {int(q4_excess.isna().any(axis=1).sum())}")
factor_stats = (ff3 * 100).describe().loc[["mean", "std", "min", "max"]]
show_table(factor_stats.rename_axis("Monthly, in %"), 2)
```

```
Ken French 3-factor file: 1926-07 to 2026-09
Sample used: 2016-01 to 2025-12 (120 months), months with a missing ETF return: 0
```

Monthly, in %	Mkt-RF	SMB	HML	RF
mean	1.08	−0.17	−0.04	0.17
std	4.54	2.77	3.85	0.16
min	−13.37	−5.93	−13.83	0.00
max	13.60	7.14	12.86	0.48

```
from scipy import stats

q4_assets = config["q4_assets"]
factor_names = ["Mkt-RF", "SMB", "HML"]
capm_fit = {a: ols(q4_excess[a], ff3["Mkt-RF"]) for a in q4_assets}
ff3_fit = {a: ols(q4_excess[a], ff3[factor_names]) for a in q4_assets}


def with_t(fit, name, scale=1):
    """Coefficient (times scale) with its t-statistic in parentheses."""
    text = f"{fit.params[name] * scale:.2f} ({fit.tvalues[name]:.2f})"
    return text.replace("-", "\N{MINUS SIGN}")


def compare_models(asset):
    """What changes for one asset when SMB and HML are added to the CAPM."""
    capm, ff = capm_fit[asset], ff3_fit[asset]
    alpha_capm, alpha_ff = capm.params["const"] * 1200, ff.params["const"] * 1200
    return {
        r"CAPM $\hat\beta_M$": capm.params["Mkt-RF"],
        "$R^2$ CAPM": capm.rsquared,
        "$R^2$ FF3": ff.rsquared,
        # = (R2_FF3 - R2_CAPM)/(1 - R2_CAPM)
        "Var($e$) share": 1 - ff.resid.var() / capm.resid.var(),
        r"$\Delta\hat\alpha$": f"{alpha_ff - alpha_capm:.2f}",
        r"$\hat\alpha$ share": 1 - alpha_ff / alpha_capm,
    }


# Annualized alpha = monthly intercept x 12, in % per year; t-statistics test H0: 0
estimates = pd.DataFrame({a: {
    r"CAPM $\hat\alpha$ (t)": with_t(capm_fit[a], "const", 1200),
    r"FF3 $\hat\alpha$ (t)": with_t(ff3_fit[a], "const", 1200),
    r"$\hat\beta_M$ (t)": with_t(ff3_fit[a], "Mkt-RF"),
    r"$\hat\beta_{\mathrm{SMB}}$ (t)": with_t(ff3_fit[a], "SMB"),
    r"$\hat\beta_{\mathrm{HML}}$ (t)": with_t(ff3_fit[a], "HML"),
} for a in q4_assets}).T
comparison = pd.DataFrame({a: compare_models(a) for a in q4_assets}).T

n_obs, df_ff3 = int(ff3_fit["IWD"].nobs), int(ff3_fit["IWD"].df_resid)
t_crit = stats.t.ppf(0.975, df_ff3)
print(f"{n_obs} months; 5% two-sided critical value of |t| with {df_ff3} residual "
      f"degrees of freedom (FF3): {t_crit:.2f}")
print("Annualized alpha (% per year) under both models, and the FF3 loadings:")
```

```
show_table(estimates.rename_axis("Asset"), 2)
print("CAPM -> FF3: R-squared, share of the CAPM residual variance explained by SMB "
      "and HML,\nchange in annualized alpha (percentage points) and share of the "
      "CAPM alpha removed:")
show_table(comparison.rename_axis("Asset"), 3)
```

```
120 months; 5% two-sided critical value of |t| with 116 residual degrees of freedom
(FF3): 1.98
Annualized alpha (% per year) under both models, and the FF3 loadings:
```

Asset	CAPM $\hat{\alpha}$ (t)	FF3 $\hat{\alpha}$ (t)	$\hat{\beta}_M$ (t)	$\hat{\beta}_{\text{SMB}}$ (t)	$\hat{\beta}_{\text{HML}}$ (t)
IWD	−2.74 (−1.42)	−2.60 (−2.35)	0.91 (43.47)	−0.02 (−0.49)	0.37 (15.81)
IWF	2.45 (1.42)	1.67 (1.85)	1.07 (63.31)	−0.13 (−4.50)	−0.32 (−17.13)
IWN	−6.05 (−1.66)	−1.83 (−2.12)	0.98 (60.47)	0.82 (30.77)	0.55 (30.08)
IWO	−6.49 (−2.24)	−2.31 (−1.81)	1.04 (43.07)	0.89 (22.45)	−0.05 (−1.74)
AAPL	11.58 (1.69)	10.43 (1.57)	1.17 (9.41)	−0.18 (−0.89)	−0.48 (−3.48)
GLD	11.91 (2.59)	10.15 (2.24)	0.13 (1.49)	−0.36 (−2.57)	−0.08 (−0.88)

```
CAPM -> FF3: R-squared, share of the CAPM residual variance explained by SMB and HML,
change in annualized alpha (percentage points) and share of the CAPM alpha removed:
```

Asset	CAPM $\hat{\beta}_M$	R^2 CAPM	R^2 FF3	Var(e) share	$\Delta\hat{\alpha}$	$\hat{\alpha}$ share
IWD	0.905	0.853	0.954	0.684	0.14	0.051
IWF	1.045	0.906	0.975	0.740	−0.79	0.321
IWN	1.157	0.726	0.985	0.946	4.22	0.697
IWO	1.220	0.823	0.967	0.813	4.18	0.644
AAPL	1.133	0.418	0.479	0.104	−1.15	0.099
GLD	0.053	0.003	0.067	0.063	−1.76	0.148

```
style_etfs = ["IWD", "IWF", "IWN", "IWO"]
for model, fits in [("CAPM", capm_fit), ("FF3", ff3_fit)]:
    residuals = pd.DataFrame({a: fits[a].resid for a in style_etfs})
    print(f"Residual correlations of the style ETFs under the {model}:")
    show_table(residuals.corr().rename_axis(f"{model} residuals"), 3)

# A correlation r estimated from n months has t = r sqrt(n - 2)/sqrt(1 - r^2)
r_crit = t_crit / np.sqrt(n_obs - 2 + t_crit**2)
print(f"With {n_obs} months, |r| > {r_crit:.2f} is different from zero at the 5% level")
```

```
Residual correlations of the style ETFs under the CAPM:
```

CAPM residuals	IWD	IWF	IWN	IWO
IWD	1.000	−0.935	0.635	0.048
IWF	−0.935	1.000	−0.774	−0.283
IWN	0.635	−0.774	1.000	0.619
IWO	0.048	−0.283	0.619	1.000

```
Residual correlations of the style ETFs under the FF3:
```

FF3 residuals	IWD	IWF	IWN	IWO
IWD	1.000	−0.878	0.496	0.258
IWF	−0.878	1.000	−0.340	−0.374
IWN	0.496	−0.340	1.000	0.131
IWO	0.258	−0.374	0.131	1.000

```
With 120 months, |r| > 0.18 is different from zero at the 5% level
```

```
x = q4_excess  # the risk-free rate cancels in every long-short spread below
spreads = pd.DataFrame({
    "HML_ETF": (x["IWN"] + x["IWD"]) / 2 - (x["IWO"] + x["IWF"]) / 2,
    "HML (Ken French)": ff3["HML"],
    "IWD - IWF": x["IWD"] - x["IWF"],
    "IWN - IWO": x["IWN"] - x["IWO"],
})
spread_stats = pd.DataFrame({
    "Mean, % p.a.": spreads.mean() * 1200,
    "t(mean)": spreads.mean() / spreads.sem(),
    "Corr. with HML": spreads.corrwith(ff3["HML"]),
})
show_table(spread_stats.rename_axis("Long-short return"), 2)

# Sources of HML_ETF's mean: alpha + sum over factors of loading x factor mean
hml_etf_fit = ols(spreads["HML_ETF"], ff3[factor_names])
factor_means = ff3[factor_names].mean() * 1200
sources = {f: [with_t(hml_etf_fit, f), factor_means[f],
               hml_etf_fit.params[f] * factor_means[f]] for f in factor_names}
sources["Alpha, % p.a."] = [with_t(hml_etf_fit, "const", 1200), "",
                            hml_etf_fit.params["const"] * 1200]
sources["Mean of HML_ETF"] = ["", "", spreads["HML_ETF"].mean() * 1200]
columns = ["Estimate (t)", "Factor mean, % p.a.", "Contribution, % p.a."]
print(f"FF3 regression of HML_ETF (R-squared {hml_etf_fit.rsquared:.3f}):")
show_table(pd.DataFrame.from_dict(sources, orient="index", columns=columns)
           .rename_axis("Term"), 2)
```

Long-short return	Mean, % p.a.	t(mean)	Corr. with HML
HML_ETF	−3.69	−1.24	0.91
HML (Ken French)	−0.49	−0.12	1.00
IWD - IWF	−7.01	−1.97	0.83
IWN - IWO	−0.38	−0.13	0.87

`FF3 regression of HML_ETF (R-squared 0.854):`

Term	Estimate (t)	Factor mean, % p.a.	Contribution, % p.a.
Mkt-RF	−0.11 (−4.91)	12.95	−1.44
SMB	0.02 (0.57)	−2.07	−0.04
HML	0.64 (25.46)	−0.49	−0.32
Alpha, % p.a.	−1.90 (−1.58)		−1.90
Mean of HML_ETF			−3.69

Task to Student:

- **Do the signs match the labels and my prediction?** With 116 residual degrees of freedom, a loading is significant at 5% when $|t| > 1.98$. All eight estimated signs match my prediction in (b), and six of the eight loadings are significant. Size: `IWN` ($\hat{\beta}_{\text{SMB}} = 0.82, t = 30.77$) and `IWO` ($0.89, t = 22.45$) load heavily on SMB, inside the 0.8 to 1 range I predicted for `IWN`; `IWF` is negative ($-0.13, t = -4.50$); `IWD` is -0.02 ($t = -0.49$), negative but statistically zero, which is the "only slightly negative" I expected from a Russell 1000 fund. Value: `IWD` ($\hat{\beta}_{\text{HML}} = 0.37, t = 15.81$) and `IWN` ($0.55, t = 30.08$) are positive and `IWF` is negative ($-0.32, t = -17.13$). The surprise is `IWO`: $\hat{\beta}_{\text{HML}} = -0.05$ ($t = -1.74$) has the predicted sign but is not significantly different from zero, so the clearly negative loading I predicted for a "growth" fund is not there. My rule that labels map directly into signs was too simple: a label ranks a fund within its own universe, while a loading is measured against the whole market. The value-minus-growth gap in $\hat{\beta}_{\text{HML}}$ is similar for small caps ($0.55 - (-0.05) = 0.60$) and large caps ($0.37 - (-0.32) = 0.69$), but the small-cap pair is centred near 0.25 and the large-cap pair near zero, so the growth half of the Russell 2000 ends up roughly HML-neutral.
- **Which APT assumption fails, and how much of `IWN`'s residual is factor risk?** The APT assumes that, once the common factors are removed, the residuals are asset-specific: $\text{Cov}(e_i, e_j) = 0$ for $i \neq j$, so they diversify away and earn no premium. The CAPM residuals of the style ETFs violate this. With 120 months a correlation is significant at 5% when $|r| > 0.18$, and five of the six pairs pass that bar: `IWD` and `IWF` -0.935, `IWF` and `IWN` -0.774, `IWD` and `IWN` 0.635, `IWN` and `IWO` 0.619, `IWF` and `IWO` -0.283. Value moves with value, small with small, and value against growth, so the CAPM has left common factors inside what it calls idiosyncratic risk. For `IWN`, the CAPM R^2 of 0.726 labels 27.4% of the variance asset-specific; SMB and HML explain 0.946 of that residual variance, which raises R^2 to 0.985. So about 26% of `IWN`'s total variance is systematic size and value risk, and only 1.5% remains asset-specific. The FF3 does not fully restore the assumption: `IWN` and `IWO` fall to 0.131, but the other five pairs still exceed 0.18 and `IWD` and `IWF` stay at -0.878. Part of that is by construction, because the two funds split the Russell 1000, which is close to the market portfolio, so their residuals must nearly offset each other.
- **How much alpha does the FF3 explain, and why is HML^{ETF}'s mean so low?** The CAPM alphas are -6.05% a year for `IWN` ($t = -1.66$) and -6.49% for `IWO` ($t = -2.24$), so "about -6%" holds in my data, though only `IWO`'s is significant at 5%. The FF3 cuts them to -1.83% ($t = -2.12$) and -2.31% ($t = -1.81$), removing 4.22 and 4.18 points, or 70% and 64%. What the CAPM called pricing error was mostly size risk: both funds load heavily on SMB (0.82 and 0.89), which earned -2.07% a year, and their market

betas fall from 1.157 and 1.220 to 0.98 and 1.04. The FF3 does not explain GLD: its alpha only falls from 11.91% ($t = 2.59$) to 10.15% ($t = 2.24$) and R^2 reaches just 0.067, so gold's return comes from outside the equity factors. HML^{ETF} tracks HML (correlation 0.91) but averages -3.69% a year against -0.49%. Its FF3 regression splits that mean into -0.32 from an HML loading of only 0.64, -1.44 from a market beta of -0.11 while the market earned 12.95%, and -1.90 of alpha ($t = -1.58$). The loading is below one because the Russell indexes split each universe roughly in half, so the ETFs hold many middle stocks that HML's extreme 30% groups exclude. Dilution alone would pull the mean towards zero; the shortfall comes from the large-cap leg (IWD minus IWF averages -7.01%, IWN minus IWO -0.38%), where the growth fund has the higher market beta and the broad value half lagged (IWD's FF3 alpha is -2.60%, $t = -2.35$). Fees are similar on both legs and largely cancel.

5. Adding profitability and investment: the Fama-French five-factor model (20 marks)

$$X_{i,t} = \alpha_i + \beta_{i,M} X_{M,t} + \beta_{i,\text{SMB}} \text{SMB}_t + \beta_{i,\text{HML}} \text{HML}_t + \beta_{i,\text{RMW}} \text{RMW}_t + \beta_{i,\text{CMA}} \text{CMA}_t + e_{i,t}.$$

Task to AI:

```
ff5_all = get_ff_factors("F-F_Research_Data_5_Factors_2x3")
ff5 = ff5_all.loc[ff_start:ff_end]
factors5 = ["Mkt-RF", "SMB", "HML", "RMW", "CMA"]
q5_assets = config["q5_assets"]
q5_excess = monthly_excess_ff(q5_assets, ff5_all["RF"])

print(f"Ken French 5-factor file: {ff5_all.index[0]} to {ff5_all.index[-1]}")
print(f"Sample used: {ff5.index[0]} to {ff5.index[-1]} ({len(ff5)} months), "
      f"months with a missing ETF return: {int(q5_excess.isna().any(axis=1).sum())}")
print(f"Correlation of SMB with HML: {ff3['SMB'].corr(ff3['HML']):.2f} in the "
      f"3-factor file, {ff5['SMB'].corr(ff5['HML']):.2f} in the 5-factor file")

# the same factor in the two files, over the sample months
files = pd.DataFrame({"Correlation": ff5[ff3.columns].corrwith(ff3),
                      "Largest gap, pp": (ff5[ff3.columns] - ff3).abs().max() * 100})
show_table(files.rename_axis("5- vs. 3-factor file"))
```

```
Ken French 5-factor file: 1963-07 to 2026-09
Sample used: 2016-01 to 2025-12 (120 months), months with a missing ETF return: 0
Correlation of SMB with HML: 0.09 in the 3-factor file, 0.34 in the 5-factor file
```

5- vs. 3-factor file	Correlation	Largest gap, pp
Mkt-RF	1.000	0.040
SMB	0.958	3.490
HML	1.000	0.000
RF	1.000	0.000

```
ff3_factors = ["Mkt-RF", "SMB", "HML"]
designs = {"CAPM": ff3[["Mkt-RF"]], "FF3": ff3[ff3_factors], "FF5": ff5[factors5]}
fits = {model: {a: ols(q5_excess[a], X) for a in q5_assets}
        for model, X in designs.items()}
```

```
# Extra step between the FF3 and the FF5: the FF3 with the five-factor file's SMB.
# It separates the effect of the new SMB series from that of adding RMW and CMA.
steps = {"FF3": fits["FF3"],
         "FF3 with the FF5 SMB": {a: ols(q5_excess[a], ff5[ff3_factors])
                                  for a in q5_assets},
         "FF5": fits["FF5"]}


def coef_t(fit, name, scale=1):
    """Coefficient (times `scale`) with its t-statistic for H0: 0 in parentheses."""
    text = f"{fit.params[name] * scale:.2f} ({fit.tvalues[name]:.2f})"
    return text.replace("-", "−")  # typographic minus sign


def by_asset(models, stat):
    """One row per asset and one column per model, filled with stat(fit)."""
    return pd.DataFrame({model: {a: stat(fit) for a, fit in by.items()}
                         for model, by in models.items()})


# annualized alpha in percent = monthly intercept x 12 x 100
alphas = by_asset(fits, lambda fit: coef_t(fit, "const", 1200))
show_table(alphas.rename_axis("Alpha, % p.a. (t)"))

loadings = pd.DataFrame({k: {a: coef_t(fit, k) for a, fit in fits["FF5"].items()}
                         for k in factors5})
show_table(loadings.rename_axis("FF5 loading (t)"))

for k in ["SMB", "HML"]:
    table = by_asset(steps, lambda fit: coef_t(fit, k))
    show_table(table.rename_axis(f"{k} loading (t)"))

adj_r2 = by_asset(fits, lambda fit: fit.rsquared_adj)
adj_r2["FF5 − FF3"] = adj_r2["FF5"] - adj_r2["FF3"]
adj_r2["Months"] = [int(fit.nobs) for fit in fits["FF5"].values()]
show_table(adj_r2.rename_axis("Adjusted $R^2$"))
```

Alpha, % p.a. (t)	CAPM	FF3	FF5
IWM	−6.19 (−2.11)	−2.01 (−2.49)	−1.52 (−2.36)
IJR	−5.55 (−1.69)	−1.41 (−1.42)	−1.43 (−1.59)
ARKK	−4.96 (−0.63)	0.04 (0.01)	2.68 (0.48)
QUAL	−0.52 (−0.47)	−1.33 (−1.36)	−1.68 (−1.86)
IWO	−6.49 (−2.24)	−2.31 (−1.81)	−1.54 (−1.49)

FF5 loading (t)	Mkt-RF	SMB	HML	RMW	CMA
IWM	1.00 (78.67)	0.81 (36.54)	0.11 (5.79)	−0.13 (−4.80)	−0.04 (−1.53)
IJR	0.96 (54.26)	0.87 (28.39)	0.21 (7.73)	0.12 (3.14)	0.02 (0.47)
ARKK	1.68 (15.33)	0.59 (3.11)	−0.72 (−4.31)	−1.32 (−5.79)	−0.39 (−1.58)
QUAL	0.98 (54.98)	−0.08 (−2.60)	−0.02 (−0.86)	0.21 (5.61)	−0.02 (−0.38)
IWO	1.03 (50.51)	0.80 (22.53)	−0.15 (−4.86)	−0.26 (−5.99)	−0.08 (−1.65)

SMB loading (t)	FF3	FF3 with the FF5 SMB	FF5
IWM	0.85 (34.17)	0.86 (39.20)	0.81 (36.54)
IJR	0.83 (26.94)	0.83 (28.99)	0.87 (28.39)
ARKK	1.17 (6.05)	1.11 (5.66)	0.59 (3.11)
QUAL	−0.17 (−5.57)	−0.15 (−4.93)	−0.08 (−2.60)
IWO	0.89 (22.45)	0.90 (24.41)	0.80 (22.53)

HML loading (t)	FF3	FF3 with the FF5 SMB	FF5
IWM	0.24 (14.46)	0.07 (4.41)	0.11 (5.79)
IJR	0.40 (19.52)	0.24 (11.37)	0.21 (7.73)
ARKK	−0.90 (−6.88)	−1.12 (−7.88)	−0.72 (−4.31)
QUAL	−0.02 (−1.09)	0.01 (0.33)	−0.02 (−0.86)
IWO	−0.05 (−1.74)	−0.23 (−8.56)	−0.15 (−4.86)

Adjusted R^2	CAPM	FF3	FF5	FF5 − FF3	Months
IWM	0.812	0.986	0.991	0.005	120
IJR	0.764	0.979	0.983	0.004	120
ARKK	0.589	0.749	0.802	0.053	120
QUAL	0.953	0.963	0.969	0.006	120
IWO	0.822	0.966	0.978	0.012	120

```
correlations = ff5[factors5].corr().round(2) + 0  # "+ 0" turns -0.00 into 0.00
show_table(correlations.rename_axis("Correlation"), 2)
```

Correlation	Mkt-RF	SMB	HML	RMW	CMA
Mkt-RF	1.00	0.34	0.01	0.00	−0.22
SMB	0.34	1.00	0.34	−0.32	0.04
HML	0.01	0.34	1.00	0.15	0.63
RMW	0.00	−0.32	0.15	1.00	0.20
CMA	−0.22	0.04	0.63	0.20	1.00

```
windows = {"1963-07 to 2013-12": ("1963-07", "2013-12"),
           "2014-01 to 2025-12": ("2014-01", "2025-12")}
spanning = {}
for label, (first, last) in windows.items():
    f = ff5_all.loc[first:last, factors5]
    std_error = f.std() / np.sqrt(len(f))  # of the monthly mean
    show_table(pd.DataFrame({
        "Mean, % p.a.": f.mean() * 1200,
        "Volatility, % p.a.": f.std() * np.sqrt(12) * 100,
        "Std. error, % p.a.": std_error * 1200,
        "t-statistic": f.mean() / std_error,
    }).rename_axis(label), 2)
    fit = ols(f["HML"], f.drop(columns="HML"))
    spanning[label] = {"Months": f"{fit.nobs:.0f}",
                       "Intercept, % p.a.": fit.params["const"] * 1200,
                       "t-statistic of the intercept": fit.tvalues["const"],
                       **fit.params.drop("const").add_prefix("Slope on "),
                       "$R^2$": fit.rsquared}

show_table(pd.DataFrame(spanning).rename_axis("HML on the other four factors"), 2)
```

1963-07 to 2013-12	Mean, % p.a.	Volatility, % p.a.	Std. error, % p.a.	t-statistic
Mkt-RF	6.01	15.53	2.19	2.75
SMB	3.36	10.55	1.49	2.26
HML	4.58	9.67	1.36	3.36
RMW	3.22	7.79	1.10	2.94
CMA	3.89	6.93	0.98	3.99

2014-01 to 2025-12	Mean, % p.a.	Volatility, % p.a.	Std. error, % p.a.	t-statistic
Mkt-RF	11.85	15.06	4.35	2.72
SMB	−2.86	10.12	2.92	−0.98
HML	−1.40	12.51	3.61	−0.39
RMW	3.08	7.30	2.11	1.46
CMA	−1.43	7.91	2.28	−0.63

HML on the other four factors	1963-07 to 2013-12	2014-01 to 2025-12
Months	606	144
Intercept, % p.a.	−0.09	0.25
t-statistic of the intercept	−0.09	0.09
Slope on Mkt-RF	0.02	0.01
Slope on SMB	0.03	0.43
Slope on RMW	0.16	0.25
Slope on CMA	1.01	0.94
R^2	0.50	0.51

Task to Student:

- **Negative $\hat{\beta}_{\mathrm{RMW}}$ and the IWO puzzle.** I call a coefficient significant when $|t| > 1.98$ (5% level, 114 degrees of freedom in the FF5). In each pair the fund without a profitability screen has the negative loading: IWM −0.13 ($t = -4.80$) against IJR 0.12 ($t = 3.14$), and ARKK −1.32 ($t = -5.79$) against QUAL 0.21 ($t = 5.61$). The Russell 2000 includes many loss-making small firms, while the S&P 600 admits only firms with positive earnings; ARKK holds unprofitable innovation firms, while QUAL selects on high return on equity. The FF3 has no profitability factor, so it can only place funds by size and book-to-market: IWM and IJR get nearly the same $\hat{\beta}_{\mathrm{SMB}}$ (0.85 and 0.83) and differ only in $\hat{\beta}_{\mathrm{HML}}$ (0.24 and 0.40), as if IJR were simply more of a value fund. The puzzle is that IWO, a growth fund, has an insignificant FF3 $\hat{\beta}_{\mathrm{HML}}$ of −0.05 ($t = -1.74$). In the FF5 it is −0.15 ($t = -4.86$) next to $\hat{\beta}_{\mathrm{RMW}}$ = −0.26 ($t = -5.99$): IWO is small, low book-to-market and weakly profitable, and its alpha shrinks from −2.31% to −1.54%. RMW is only part of the resolution. The FF5 SMB alone already gives −0.23 ($t = -8.56$): the FF3 SMB behaves like the FF5 SMB with a short position in HML (correlation with HML 0.09, against 0.34), so IWO's FF3 SMB loading carried most of its growth tilt.
- **ARKK's $\hat{\beta}_{\mathrm{SMB}}$ and adjusted R^2.** ARKK's SMB loading falls from 1.17 ($t = 6.05$) in the FF3 to 0.59 ($t = 3.11$) in the FF5. The new SMB series is not the cause: with the FF5 SMB but without RMW and CMA the loading is still 1.11. The cause is omitted-variable bias. When a factor is left out, the loading on an included factor absorbs the omitted loading times the co-movement between the two factors. ARKK loads heavily and negatively on RMW (−1.32), and SMB is negatively correlated with RMW (−0.32) because small firms are on average less profitable. The product of two negatives is positive, so the FF3 overstates $\hat{\beta}_{\mathrm{SMB}}$: part of what looked like a small-cap tilt is an unprofitable-firm tilt. CMA adds little, because its correlation with SMB is 0.04 and ARKK's $\hat{\beta}_{\mathrm{CMA}}$ (−0.39, $t = -1.58$) is insignificant. Adjusted R^2 rises meaningfully only for ARKK, from 0.749 to 0.802. IWO gains 0.012 (0.966 to 0.978), small in level but about a third of the variance the FF3 left unexplained. For IWM, IJR and QUAL the gain is 0.004 to 0.006: their RMW loadings are significant, but the FF3 already explains 96% to 99% of their variance.
- **HML's intercept and the length of the sample.** The intercept of HML on the other four factors is −0.09% a year ($t = -0.09$) in 1963-2013 and 0.25% ($t = 0.09$) in 2014-2025, although HML's own mean in the first period is 4.58% ($t = 3.36$). HML's average return is therefore fully explained by its loadings on the other factors, mainly CMA (slopes 1.01 and 0.94). HML is redundant for describing average returns: adding it to the other four does not improve the best attainable Sharpe ratio. It still helps describe co-movement (R^2 is only about 0.5), which is why my ETFs load on it. All five premia are significant in 1963-2013 (t from 2.26 to 3.99), but since 2014 only the market's is, at 11.85% ($t = 2.72$); the others have $|t| \leq 1.46$. That does not show the premia are gone. The standard error of an annualized mean is about $\sigma/\sqrt{T}$ with T in years, however often returns are sampled. For HML over these 12 years it is $12.51/\sqrt{12} = 3.61\%$, so $|t| = 2$ needs a mean of 7.2%, more than the earlier 4.58%; detecting a true 4.58% would take $(2 \times 12.51/4.58)^2 \approx 30$ years. RMW's mean barely changed (3.22% to 3.08%), yet its t fell from 2.94 to 1.46. As with the rolling betas in Q3, a short window can follow a regime change but is noisy, and a long window is precise but averages over regimes.

(c) Data sources (for Q1, Q4–Q5)

- **Prices (Q1–Q3).** Yahoo Finance, downloaded with `yfinance` on 10 November 2026: daily closing prices adjusted for splits and dividends for `SPY` (market proxy), `GLD`, `GDX` and `AAPL`, and the daily settlement price of `GC=F` (Yahoo's continuous front-month COMEX gold futures series). Sample: 3 January 2005 to 31 December 2025 on NYSE trading days (the template's default window, unchanged); `GDX` starts on 23 May 2006 (it was launched that month), and `GC=F` has no quote on a few NYSE days, so each regression uses the days (or months) on which both the asset and `SPY` have a return.
- **Risk-free rate (Q1–Q3).** Yahoo Finance `^IRX`, the 13-week (3-month) US Treasury bill yield, quoted in percent per year. I divide it by 100 and then by 12 (monthly), 52 (weekly, Q2 only) or 252 (daily), and use the yield quoted at the end of the previous period as the risk-free return for the current period. Days with no `^IRX` quote take the previous quote.
- **Returns.** Simple returns from adjusted prices: daily from consecutive trading days, monthly from month-end prices (252 months, January 2005 to December 2025), weekly from Friday closes. $X_{i,t} = R_{i,t} - R_{f,t}$ and $X_{M,t} = R_{\mathrm{SPY},t} - R_{f,t}$. Q2 and Q3 use subsets of the same daily data (2018–2020 and 2016–2025).
- **Factors (Q4–Q5).** Ken French Data Library, monthly files *Fama/French 3 Factors* (July 1926 to September 2026) and *Fama/French 5 Factors (2x3)* (July 1963 to September 2026), both built from the September 2026 CRSP database; the last available month in each is September 2026. Variables: `Mkt-RF`, `SMB`, `HML`, `RMW`, `CMA` and `RF` in percent per month, converted to decimals. The SMB series differs between the two files, so FF3 regressions use the first file and FF5 regressions the second.
- **ETFs (Q4–Q5).** Yahoo Finance month-end adjusted prices for `IWD`, `IWF`, `IWN`, `IWO`, `AAPL`, `GLD`, `IWM`, `IJR`, `ARKK` and `QUAL`; monthly simple returns from January 2016 to December 2025 (120 months) minus the Ken French `RF` (the one-month Treasury bill return). In these questions the market factor is `Mkt-RF`, not `SPY`. The long-run factor tests in Q5 use July 1963 to December 2013 and January 2014 to December 2025.
- **Reproducibility.** The first run saves every download in a `data/` folder next to this file and later runs read from it, so the numbers quoted in the text match the tables.

Render and submit

From the folder containing this file, run (Typst produces the PDF; no LaTeX needed):

```
quarto render studentID_assignment_two.qmd --to typst
```

Open the PDF and check that every table, figure and equation appears and no placeholder is left. Submit **two files**: `studentID_assignment_two.qmd` and `studentID_assignment_two.pdf`.